

Grammy Performance Planning

Introduction

Lets imagine the Grammys have had some horrible misplanning issues regarding their performance schedule. You, as a senior software engineer working for the Grammys, have now been prompted to build a software solution which is suitable for planning and managing all Grammy Awards performances of a single night, so next year troubles can be saved. Grammy Award Performances work as follows:

- A lot of the nominated artists will perform, at the stage where their category is being announced, **exactly once**(never twice on the same night), but some will not.
- Categories have different **priority levels** and artists must be present at the highest priority category they are nominated for. Thus they must not perform at a lower priority categories stage if they are nominated for a higher priority category taking place on another stage.
- Artists in the same category are given a certain **budget** for their performance (although they can consume more money if they pay for it themselves).

Data Model

The already given data model you shall use to solve the problem is as follows:

- **Stage**
 - Represents a performance stage.
 - Properties:
 - **Id** (**Guid**): Unique identifier for the stage.
 - **Name** (**string**): Name of the stage.
 - **Categories** (**ICollection<Category>**): Categories being announced at this stage.
- **Category**
 - Represents an award category.
 - Properties:
 - **Id** (**Guid**): Unique identifier for the category.
 - **Name** (**string**): Name of the category.
 - **Priority** (**PriorityLevel**): Priority, either "GenreSpecific" or "AcrossGenres".
 - **Budget** (**decimal**): Performance budget for this category.
 - **Stage** (**Stage**): The stage on which this category will be announced.
 - **Artists** (**ICollection<Artist>**): Artists nominated in this category.
 - **Winner** (**Artist?**): The winning artist of this category. This is nullable, because of security reasons it will only be set after the grammy's took place and the winners are known.
- **Artist**
 - Represents a performing or nominated artist.
 - Properties:
 - **Id** (**Guid**): Unique identifier for the artist.

- **Name** (**string**): Name of the artist.
- **Performance** (**Performance?**): The performance of the artist, if any.
- **Categories** (**ICollection<Category>**): Categories in which the artist is nominated.
- **WinningCategories** (**ICollection<Category>**): Categories the artist has won.

- **Performance**

- Represents an artist's performance at the event.
- Properties:
 - **Id** (**Guid**): Unique identifier for the performance.
 - **Artist** (**Artist**): The performing artist.
 - **Category** (**Category**): The category of the performance.
 - **DateTime** (**DateTime**): Date and time of the performance. Performances on the same stage must be at least 30 minutes apart.

- **PriorityLevel** (**enum**)

- Enumerates the priority of categories.
 - **GenreSpecific** (value: 1): Regular category, specific to a genre.
 - **AcrossGenres** (value: 2): High priority, applies across genres.

These entities together model the relationships between Grammy stages, categories, artists, winners, and actual performances, allowing the planning logic to respect both scheduling and artist-category constraints.

Importer

Since the Grammys already exists for a long time, they have developed their own **Data-Format** to organize their data. Its specification is as follows:

Example

```
A-Stage
---
Record of the year; MultiGenre; $100,000
==Tyler the Creator; $150,000; 2026-02-01T17:30:00Z==
Bad Bunny
==Clipse; $70,000; 2026-02-01T18:00:00Z==
Billie Eilish
===
Vocal album of the year; MultiGenre; $80,000
==Billie Eilish; $90,000; 2026-02-01T18:30:00Z==
==Justin Bieber; $10,000; 2026-02-01T20:30:00Z==
Doja Cat
===
Rap album of the year; SingleGenre; $50,000
Kendrick Lamar
==JID; $60,000; 2026-02-01T19:00:00Z==
Tyler the Creator
```

File Format Specification

The first line is the stage header containing the stage name.

- Stage name (mandatory, e.g., "A-Stage")

The second line is a delimiter indicating the start of stage content.

- Must be exactly `---`

Subsequent lines represent categories and their nominated artists.

Each category block consists of:

1. **Category definition line** with semicolon-separated fields:

- Category name (mandatory, e.g., "Record of the year")
- Priority level (mandatory): Either `MultiGenre` (maps to `AcrossGenres` priority) or `SingleGenre` (maps to `GenreSpecific` priority)
- Category budget (mandatory, format: `$XXX,XXX` - note the semicolon at the end)

2. **Artist lines** (one or more):

- **Performing artists:** Wrapped in `==` delimiters with format `==Artist Name; $Budget==`
 - Artist name (mandatory)
 - Performance budget (mandatory, format: `$XXX,XXX`)
 - These artists WILL perform at this category on this stage
- **Non-performing artists:** Just the artist name
 - Artist name (mandatory)
 - These artists are nominated but will NOT perform for this category

3. **Category delimiter line:**

- Must be exactly `===` to mark the end of the current category

Restrictions

When implementing the importer mind following restrictions:

- An artist can only perform once for the whole event.
- Two performances cannot take place at the same time on the same stage. Performances on the same stage must be at least 30 minutes apart.

File Validation

You must implement logic to parse and validate a Grammy Data File. Distinguish between the following validation errors:

- **Empty file**
- **Invalid stage definition**
 - Missing stage name
 - Missing `---` delimiter after stage name

- **Invalid category definition**
 - Invalid number of fields (must have exactly 3 fields separated by semicolons)
 - Empty category name
 - Invalid priority level (must be **MultiGenre** or **SingleGenre**)
 - Invalid budget format or value (must be > 0)
- **Invalid artist entry**
 - Empty artist name
 - Invalid performing artist format (if using == delimiters, must include budget)
 - Invalid performance budget (must be > 0)
 - Performance date and time is not at least 30 minutes apart from other performances on the same stage
- **Missing category delimiter** (missing === at the end of a category)
- **Priority constraint violation**
 - An artist performing at a lower priority category while nominated in a higher priority category on a different stage

Parsing Tests

Add at least 3 tests to verify the importer works correctly.

Calculation Logic

To plan on which artists to consider for next year's Grammy Award Performances, you are prompted to provide a statistic which calculates a **performance score** for each artist. The performance score is calculated by taking the sum of the following factors and dividing it by the total number of factors across all performing artists:

- The artist's **budget**
 - Exceedances of the artist's budget are considered positively, because we do save money by not having to pay for performance extras. Thus for each exceedence in 25% steps of the artist's budget, the performance score is increased by 1 point. This is capped at 10 points.
 - Undershoots of the artist's budget are considered negatively, because the artists does not use the money given to them and so on we lose money. Thus for each undershoot in 25% steps of the artist's budget, the performance score is decreased by 1 point. This is capped at -10 points.
- The artist's **winning categories**
 - The more winning categories the artist has, the higher the performance score.
 - A winning category in a across genres category is worth 2 points, a winning category in a genre specific category is a single point.
- The artist's **nominated categories**
 - The more nominated categories the artist has, the higher the performance score.
 - A nominated category in a across genres category is worth 0.25 points, a nominated category in a genre specific category is worth 0.1 points.

Mind: A **non-performing artist** is not considered for the performance score and thus not included in the calculation.

Web API

The Web API provides endpoints for retrieving Grammy performance data and setting category winners. All data creation is handled by the importer; the API focuses on read operations and winner assignment. The endpoints are as follows:

Endpoint	Method	Description
/stages	GET	Retrieve all stages
/stages/:id	GET	Retrieve a specific stage by ID
/categories	GET	Retrieve all categories. Filter by stage using query parameter
/categories/:id	GET	Retrieve a specific category by ID
/categories/:id/winner	PATCH	Set the winner for a specific category
/artists	GET	Retrieve all artists. Filter by category using query parameter
/performances	GET	Retrieve all performances. Filter by artist using query parameter
/performances/:id	GET	Retrieve a specific performance by ID
/statistics	GET	Calculate and retrieve performance scores for all performing artists. If no winner is yet announced, this shall not return any scores.

Integration Tests

Add a few (at least 3) meaningful integration tests, so we can be sure the statistics work correctly.

Frontend

Technologies

- Hosting: Aspire
- Backend: .NET
 - ASP.NET Core Web API
 - Entity Framework Core with SQLite
- Frontend: Angular 21
 - Standalone components
 - Signals
 - Generated API client using OpenAPI (`npm run generate-web-api` in the Frontend project)

Grading Criteria

Following criteria is crucially, the rest is also important tho.

- Compiles without errors
- Importer works and throws correct errors.
- All API-Queries must be implemented.
- The following frontend features are implemented:
 - Stages are queried and displayed.

- Statistics are calculated and displayed.